

12. Pruebas y calidad

PDF Reader v0.3.3 | Análisis 2026-09-29 | Referencia del release v0.3.3

Contenido

- Gates actuales
- Resultado actual
- Validación histórica de lectura
- Ejecución preliminar anterior
- Casos cubiertos
- Cobertura faltante priorizada
- Análisis estático y formato
- Criterios de aceptación
- Datos de prueba

Gates actuales

Gate	Herramienta	Cobertura observable
Unitarias y contratos Web	Node --test	29 casos de utilidades, interacción, ciclo de vida y almacenamiento defensivo
Contrato repositorio	verify-repo.mjs	estructura, alcance read-only, zoom, versión, permisos y workflows
Build Web/Pages	scripts Node	copia PDF.js y composición landing+demo
Android unit	Gradle/JUnit	prueba trivial de plantilla
Android instrumentada	AndroidX	application ID
CI Web	Ubuntu	install, check, audit, build y artifact
CI Android	Ubuntu/JDK/SDK	sync limpio, tests, APK debug y metadatos/permisos
CI Windows	Windows	instalador NSIS y portable
Release	Ubuntu+Windows	versión, firma APK, contenido y checksums
Documentación integral	verify-system-docs.mjs	20 Markdown/PDF, enlaces, tablas y UTF-8

Resultado actual

El 2026-09-28/29, para el release v0.3.3 creado desde el commit 366a27ca6cceedf3599758a872cc920e4f939850, se obtuvo:

Bloque de código

```
pnpm check                -> 29/29 Node; verify y 20 Markdown/20 PDF correctos
pnpm qa:desktop-e2e        -> 47/47 Electron; repetido sobre Portable y Setup públicos
pnpm qa:web-e2e            -> 21/21 Edge/Chromium localhost a 390x844
pnpm qa:android-e2e        -> 21/21 APK debug en Android Emulator API 36.1
APK firmado publicado      -> PASS black-box crítico tras instalación limpia
CI 36514689572             -> Web, Android y Windows PASS
Pages 36514689616         -> PASS
Release 36514950301        -> contrato, firma, paquetes y publicación PASS
```

Las suites Web y Android recorren las 16 variantes PDF, acciones compuestas, errores recuperables, historial y paneles. Android añade pinza, doble toque, swipe, orientación y background/foreground mediante la APK instalada y DocumentsUI. El detalle actual está en `reports/pdf-functional-validation-2026-09-28.md`; la prueba en

teléfono físico continúa **NO EJECUTADA**.

Validación histórica de lectura

El 2026-09-24 se ejecutaron 24/24 pruebas Node y un recorrido Web interactivo con PDF de 1, 12 y 1.000 páginas. Esa evidencia histórica, incluidos navegación, búsqueda, error recuperable y viewport móvil, permanece en `reports/functional-reading-validation-2026-09-24.md`.

Ejecución preliminar anterior

El 2026-09-23 se aplicó la suite preliminar de producto sobre Web local. La ejecución fresca posterior a las correcciones obtuvo:

Bloque de código

```
CI=true pnpm test          -> 21/21 pruebas
CI=true pnpm release:check -> test, verify, docs y build Pages correctos
pnpm build:web            -> dist generado
```

La prueba funcional abrió un PDF desde Historial y luego un segundo PDF sin recargar. Antes de la corrección el segundo paso fallaba con `previousDoc.destroy is not a function`; después, el segundo documento quedó visible sin errores nuevos. El informe trazable es `reports/preliminary-product-validation-2026-09-23.md`.

Referencia histórica de la documentación integral

El 2026-09-10 se ejecutó:

Bloque de código

```
CI=true pnpm install --frozen-lockfile -> correcto, lockfile sin cambios
pnpm check                             -> 9/9 pruebas, verify OK
pnpm build:pages                       -> dist y pages-dist generados
pnpm docs:check                       -> estructura, enlaces, tablas, UTF-8 y PDF correctos
pnpm exec node scripts/run-gradle.mjs test -> BUILD SUCCESSFUL, 138 tareas
```

El primer intento sin `CI=true` no ejecutó tests porque pnpm no podía confirmar de forma interactiva la recreación de `node_modules`. Gradle requirió seleccionar explícitamente el JDK 21 de Android Studio y el SDK local; con ambos activos, las variantes debug/release y sus pruebas terminaron correctamente.

Casos cubiertos

`utils.test.mjs` contiene 14 casos y valida cálculos, formato, snippets seguros, identidad, gestos, compartir, ancla de zoom, borde inicial del swipe y ventanas de miniaturas. `app-contract.test.mjs` contiene 11 casos y protege navegación, documentos grandes, segunda apertura, concurrencia, ciclo de vida, confirmación de página con Enter, IPC de compartir, CSP, render atómico, insets Android y objetivos táctiles. `storage.test.mjs` contiene 4 casos de lectura/escritura normal y degradación ante `SecurityError` o cuota. `verify-repo` confirma marcadores del layout, ausencia de funciones de edición, versión Android, permisos prohibidos, asociación PDF, release firmado, Pages y presencia de los harness E2E.

Cobertura faltante priorizada

- 1 **Alta:** ejecutar en CI las suites E2E Web/Electron/Android que hoy son reproducibles pero locales.
- 2 **Alta:** repetir Android en teléfono físico: intent, pinza, rotación, background/restore, compartir y tamaños de pantalla reales.
- 3 **Media:** IndexedDB: poda a ocho, transacciones fallidas, cuota y migraciones; `localStorage` bloqueado sí dispone de regresiones unitarias.
- 4 **Media:** medir memoria y tiempos con PDF pesados/alta resolución; contraseña, ausencia de OCR, corrupción recuperable y 1.000 páginas ya tienen regresión funcional.

- 5 **Media:** accesibilidad automatizada y manual con lector de pantalla.
- 6 **Baja:** reemplazar `additionIsCorrect` por pruebas del plugin nativo.

Análisis estático y formato

No se identificaron ESLint, Prettier, TypeScript, cobertura, SAST ni escáner de secretos dedicados. Node interpreta ESM/CJS durante tests/build. Los workflows ejecutan `pnpm audit --prod --audit-level high`; Dependabot revisa npm y Actions semanalmente.

Criterios de aceptación

El `spec/spec.md` exige tests Node, verificación, build Web, packaging Windows, APK debug inspeccionado, APK release firmado, documentación, Pages y release por tag. Para considerar una versión publicable deben estar verdes los tres checks protegidos de main: `Web · tests + landing`, `Windows · package` y `Android · APK debug verificado`.

Datos de prueba

No se versionan PDF privados. `scripts/create-smoke-pdf.mjs` crea PDF sintéticos deterministas de 1 a 2.000 páginas; las salidas viven en `reports/smoke/` y están ignoradas por Git. Los informes `reports/validation-v0.*.md` conservan evidencia histórica, no sustituyen una ejecución actual.